Subject 6: LLM Observability & Debugging (Tracing)

1. The 80/20 Core (Hiring-Critical)

What Actually Matters (20% → 80% Impact)

Included (Production & Interview Critical):

1. **Distributed Tracing for LLM Pipelines** (35% of value)
   - Span-based tracking (retrieval → generation → post-processing)
   - Parent-child span relationships
   - Trace visualization and analysis
   - Latency waterfall diagrams
   - Why: Multi-step LLM pipelines are black boxes without tracing; 70% of production issues require trace-level debugging
2. **Input/Output Logging with Metadata** (25% of value)
   - Capturing prompts, completions, tokens, costs
   - User context and session tracking
   - Version tagging (prompt version, model version)
   - Sampling strategies for high-volume systems
   - Why: Can't debug what you can't see; logs are ground truth for incident response
3. **Error Classification & Debugging Workflows** (20% of value)
   - Error taxonomy (rate limits, timeouts, validation failures, hallucinations)
   - Automatic error categorization
   - Playbooks per error type
   - Error rate monitoring and alerting
   - Why: Different errors need different fixes; systematic classification enables faster resolution
4. **Performance Metrics & Anomaly Detection** (10% of value)
   - Latency tracking (p50, p95, p99)
   - Token usage patterns
   - Cache hit rates
   - Quality score distributions
   - Why: Performance degradation often precedes quality issues

5. **User Feedback Integration** (10% of value)
   - Thumbs up/down tracking
   - Explicit user corrections
   - A/B test result collection
   - Feedback → trace mapping
   - Why: User feedback is the ultimate quality signal; must connect to underlying traces

Explicitly Excluded (Low ROI / Interview Anti-Signals):

- ❌ Print statements / console.log debugging
- *Why excluded:* Not scalable, disappears in production, signals amateur
- ❌ Generic application monitoring (New Relic, Datadog alone)
- *Why excluded:* Doesn't understand LLM-specific patterns (tokens, prompts, quality)
- ❌ No structured logging (unstructured text logs)
- *Why excluded:* Can't query, aggregate, or analyze; manual inspection doesn't scale
- ❌ Logging everything (no sampling strategy)
- *Why excluded:* Cost explosion, storage issues, privacy risks
- ❌ Post-hoc debugging only (no real-time monitoring)
- *Why excluded:* Incidents escalate before you notice; need proactive detection

Hiring Signal Justification

What Tier 1 interviewers look for:

- Can you trace a multi-step LLM pipeline end-to-end?
- Do you have structured logging with queryable metadata?
- Can you debug production LLM failures systematically?
- Have you implemented error alerting with actionable playbooks?
- Do you track LLM-specific metrics (token usage, quality scores)?

What signals "student not engineer":

- "I just look at the logs" (no structure, no tooling)
- No tracing instrumentation
- Manual debugging only (no automated monitoring)
- Can't explain how to debug a production LLM failure
- No understanding of sampling strategies or privacy concerns

---

2. Genius-Level Understanding

Core Mental Models

Perspective 1: Infrastructure Engineer

LLM observability is distributed tracing for non-deterministic systems.

Traditional distributed system:

```
Request → Service A → Service B → Service C → Response
          (50ms)     (100ms)     (30ms)

Total: 180ms (deterministic, reproducible)
```

LLM pipeline:

```
Query → Retrieval → Reranking → Generation → Post-processing → Response
        (50ms)      (150ms)     (800ms*)     (20ms)

Total: 1020ms (*varies: 500-2000ms, non-deterministic)
```

Key insight: LLM calls are:

- **Non-deterministic** (same input → different outputs with temperature > 0)
- **Expensive** (need cost tracking per span)
- **Quality-sensitive** (need quality metrics, not just latency)
- **Token-bound** (track tokens as first-class metric)

Production implications:

python

```
# Amateur: No tracing, just timing
import time

def rag_query(question):
    start = time.time()

    docs = retriever.search(question)
    answer = llm.generate(question, docs)
```

```
    print(f"Total time: {time.time() - start:.2f}s")  # ❌ Where was the time spent?
    return answer

# Engineer: Full distributed tracing
from opentelemetry import trace
from opentelemetry.trace import Status, StatusCode

tracer = trace.get_tracer(__name__)

def rag_query(question):
    with tracer.start_as_current_span("rag_query") as span:
        # Add metadata
        span.set_attribute("question", question)
        span.set_attribute("user_id", get_user_id())
        span.set_attribute("session_id", get_session_id())

        # Retrieval span
        with tracer.start_as_current_span("retrieval") as retrieval_span:
            docs = retriever.search(question)
            retrieval_span.set_attribute("num_docs", len(docs))
            retrieval_span.set_attribute("top_score", docs[0].score if docs else 0)

        # Generation span
        with tracer.start_as_current_span("generation") as gen_span:
            try:
                answer = llm.generate(question, docs)

                gen_span.set_attribute("model", "gpt-4")
                gen_span.set_attribute("input_tokens", count_tokens(question, docs))
                gen_span.set_attribute("output_tokens", count_tokens(answer))
                gen_span.set_attribute("cost", calculate_cost(...))

                span.set_status(Status(StatusCode.OK))
                return answer

            except Exception as e:
                gen_span.set_status(Status(StatusCode.ERROR, str(e)))
                gen_span.record_exception(e)
                raise

# Now can answer:
# - Where is latency coming from? (retrieval vs generation)
# - What was the exact input/output?
# - How much did this cost?
# - Did any errors occur?
# - What was the retrieval quality?
```

Trace visualization:

```
Trace ID: abc123
Duration: 1.02s
├─ rag_query (1.02s)
│  ├─ retrieval (0.05s) ✓
│  │  └─ [num_docs: 5, top_score: 0.87]
│  └─ generation (0.95s) ✓
│     └─ [model: gpt-4, tokens_in: 1200, tokens_out: 350, cost: $0.042]
```

Perspective 2: Product Engineer

Observability enables root cause analysis for user-facing issues.

The Debugging Funnel:

```
User Report: "Bot gave wrong answer to my question"
           ↓
1. Find the trace (by user_id, session_id, timestamp)
           ↓
2. Inspect retrieval span
   - Were relevant docs retrieved? (top_score)
   - How many docs? (num_docs)
   - What was the query?
           ↓
3. Inspect generation span
   - What prompt was sent to LLM?
   - What was the response?
   - Any errors/retries?
           ↓
4. Root cause identified:
   - Retrieval issue? → Fix chunking/indexing
   - Generation issue? → Fix prompt
   - Context overflow? → Reduce docs
```

Real example:

python

```
# User complaint: "Bot keeps saying 'I don't have information' but docs clearly exist"

# 1. Find traces for this user
traces = query_traces(
    user_id="user_123",
```

```
    timestamp_range=("2025-02-09T10:00", "2025-02-09T11:00")
)

# 2. Filter to failed cases
failed_traces = [t for t in traces if "I don't have information" in t.output]

# 3. Analyze retrieval spans
for trace in failed_traces:
    retrieval_span = trace.get_span("retrieval")

    print(f"Query: {retrieval_span.attributes['query']}")
    print(f"Num docs: {retrieval_span.attributes['num_docs']}")
    print(f"Top score: {retrieval_span.attributes['top_score']}")

# Output reveals pattern:
# All failed queries have top_score < 0.3 (low relevance)
# Root cause: Embedding model doesn't understand domain terminology
# Fix: Use domain-specific embeddings or fine-tune
```

Why this matters: Without traces, debugging is guesswork. With traces, it's systematic.

Perspective 3: Researcher

LLM debugging requires capturing non-determinism and quality, not just performance.

Advanced insight: Quality metrics are first-class observability primitives

Traditional systems track:

- Latency ✓
- Error rate ✓
- Throughput ✓

LLM systems must also track:

- **Quality scores** (hallucination rate, groundedness, relevance)
- **Token efficiency** (tokens per semantic unit)
- **Cost efficiency** (cost per quality point)
- **Variance** (output diversity across runs)

python

```
# Capture quality metrics alongside performance
class QualityInstrumentedLLM:
```

```
    def __init__(self, llm, tracer):
        self.llm = llm
        self.tracer = tracer

    def generate(self, prompt, context):
        with self.tracer.start_as_current_span("llm_generation") as span:
            # Generate
            response = self.llm.complete(prompt)

            # Standard metrics
            span.set_attribute("latency_ms", span.duration_ms)
            span.set_attribute("tokens_in", count_tokens(prompt))
            span.set_attribute("tokens_out", count_tokens(response))

            # Quality metrics (NEW!)
            span.set_attribute("groundedness_score",
                               self.measure_groundedness(response, context))

            span.set_attribute("relevance_score",
                               self.measure_relevance(response, prompt))

            span.set_attribute("hallucination_detected",
                               self.detect_hallucination(response, context))

            # Cost efficiency
            span.set_attribute("cost_per_token",
                               span.cost / span.tokens_out)

            return response

    def measure_groundedness(self, response, context):
        """Check if response is grounded in context"""
        # Use LLM-as-judge or NLI model
        claims = extract_claims(response)
        grounded = sum(1 for c in claims if claim_in_context(c, context))
        return grounded / len(claims) if claims else 1.0

    def detect_hallucination(self, response, context):
        """Binary hallucination detection"""
        groundedness = self.measure_groundedness(response, context)
        return groundedness < 0.8  # Threshold
```

Now can query:

sql

```
-- Find traces with high hallucination rates
SELECT trace_id, user_id, hallucination_detected
FROM traces
WHERE hallucination_detected = true
  AND timestamp > NOW() - INTERVAL '24 hours'
ORDER BY timestamp DESC;

-- Avg quality by model
SELECT model, AVG(groundedness_score) as avg_groundedness
FROM traces
GROUP BY model;
```

Perspective 4: Hiring Manager

Show me your production incident response using traces.

Red flags:

- "I debug by trying different prompts until it works"
- No trace storage or querying capability
- Can't reproduce production issues in development
- No monitoring dashboards
- Manual log inspection only

Green flags:

- Trace-based debugging workflow
- Automated alerting on quality/performance degradation
- Trace → feedback → improvement loop
- Sampling strategies for cost control
- Privacy-aware logging (PII masking)

Interview question: "A user reports the bot gave them incorrect information. Walk me through your debugging process."

Strong answer:

```
1. Get trace_id from user report or query by user_id/timestamp
2. Open trace in UI (e.g., Langfuse, Phoenix, Langsmith)
3. Inspect spans:
   - Retrieval: Were relevant docs retrieved?
   - Generation: What was the exact prompt and response?
```

```
4. Check quality metrics:
   - Groundedness score
   - Hallucination detection
5. Identify root cause:
   - If retrieval bad → check indexing
   - If generation bad → check prompt
6. Reproduce in dev:
   - Use same inputs from trace
   - Verify fix
7. Deploy fix, monitor metrics
```

Advanced Production Use Cases

Case 1: Multi-Tenant Quality Monitoring

Problem: Different users/organizations have different quality expectations.

Solution: Per-tenant observability with quality SLOs.

python

```
from opentelemetry import trace
import logging

class TenantAwareObservability:
    def __init__(self, tracer):
        self.tracer = tracer

        # Tenant-specific SLOs
        self.tenant_slos = {
            'enterprise_client_1': {
                'latency_p95_ms': 1000,
                'groundedness_min': 0.95,
                'cost_per_query_max': 0.05
            },
            'startup_client_2': {
                'latency_p95_ms': 3000,
                'groundedness_min': 0.85,
                'cost_per_query_max': 0.01
            }
        }

    def track_query(self, tenant_id, query, answer, metadata):
```

```
        """Track query with tenant context"""
        with self.tracer.start_as_current_span("rag_query") as span:
            # Tenant metadata
            span.set_attribute("tenant_id", tenant_id)
            span.set_attribute("tenant_tier", self.get_tier(tenant_id))

            # Metrics
            span.set_attribute("latency_ms", metadata['latency_ms'])
            span.set_attribute("groundedness", metadata['groundedness'])
            span.set_attribute("cost", metadata['cost'])

            # Check SLO violations
            slo = self.tenant_slos.get(tenant_id, {})

            if metadata['latency_ms'] > slo.get('latency_p95_ms', float('inf')):
                span.add_event("slo_violation", {
                    "type": "latency",
                    "value": metadata['latency_ms'],
                    "threshold": slo['latency_p95_ms']
                })
                self.alert_slo_violation(tenant_id, "latency")

            if metadata['groundedness'] < slo.get('groundedness_min', 0):
                span.add_event("slo_violation", {
                    "type": "quality",
                    "value": metadata['groundedness'],
                    "threshold": slo['groundedness_min']
                })
                self.alert_slo_violation(tenant_id, "quality")

# Query per-tenant metrics
def get_tenant_dashboard(tenant_id, time_range):
    traces = query_traces(
        filters={"tenant_id": tenant_id},
        time_range=time_range
    )

    return {
        'total_queries': len(traces),
        'avg_latency': np.mean([t.latency_ms for t in traces]),
        'p95_latency': np.percentile([t.latency_ms for t in traces], 95),
        'avg_groundedness': np.mean([t.groundedness for t in traces]),
        'total_cost': sum(t.cost for t in traces),
        'slo_violations': sum(1 for t in traces if has_slo_violation(t))
    }
```

Impact: Enterprise clients get guaranteed quality; can charge premium for tighter SLOs.

Case 2: A/B Test Tracking

Problem: Running multiple prompt versions simultaneously, need to compare quality.

Solution: Version tagging in traces with automatic aggregation.

python

```
class ABTestObservability:
    def __init__(self, tracer):
        self.tracer = tracer
        self.experiments = {}

    def start_experiment(self, name, variants):
        """Start A/B test"""
        self.experiments[name] = {
            'variants': variants,
            'traffic_split': {v: 1.0/len(variants) for v in variants}
        }

    def execute_with_variant(self, experiment_name, user_id, query_fn):
        """Execute query with assigned variant"""
        # Assign variant (deterministic by user_id for consistency)
        variant = self.assign_variant(experiment_name, user_id)

        with self.tracer.start_as_current_span("ab_test_query") as span:
            # Tag with experiment metadata
            span.set_attribute("experiment", experiment_name)
            span.set_attribute("variant", variant)
            span.set_attribute("user_id", user_id)

            # Execute with this variant
            result = query_fn(variant)

            # Capture metrics
            span.set_attribute("success", result.get('success', False))
            span.set_attribute("quality_score", result.get('quality', 0))
            span.set_attribute("user_rating", result.get('user_rating', 0))

            return result

    def get_experiment_results(self, experiment_name):
        """Analyze A/B test results"""
        traces = query_traces(
```

```
            filters={"experiment": experiment_name}
        )

        # Group by variant
        by_variant = defaultdict(list)
        for trace in traces:
            variant = trace.attributes['variant']
            by_variant[variant].append(trace)

        # Aggregate metrics
        results = {}
        for variant, variant_traces in by_variant.items():
            results[variant] = {
                'n': len(variant_traces),
                'success_rate': np.mean([t.success for t in variant_traces]),
                'avg_quality': np.mean([t.quality_score for t in variant_traces]),
                'avg_user_rating': np.mean([t.user_rating for t in variant_traces if t.u
                'p95_latency': np.percentile([t.latency_ms for t in variant_traces], 95)
            }

        # Statistical significance
        results['winner'] = self.determine_winner(results)

        return results

    def determine_winner(self, results):
        """Determine statistically significant winner"""
        from scipy import stats

        variants = list(results.keys())
        if len(variants) != 2:
            return None

        v1_scores = results[variants[0]]['quality_scores']
        v2_scores = results[variants[1]]['quality_scores']

        # T-test
        t_stat, p_value = stats.ttest_ind(v1_scores, v2_scores)

        if p_value < 0.05:  # Significant
            return variants[0] if np.mean(v1_scores) > np.mean(v2_scores) else variants[

        return None  # No significant difference

# Usage
ab_test = ABTestObservability(tracer)
```

```
# Start experiment
ab_test.start_experiment(
    name="prompt_style_test",
    variants=["formal", "casual"]
)

# Execute queries
for user_id, query in user_queries:
    result = ab_test.execute_with_variant(
        "prompt_style_test",
        user_id,
        lambda variant: rag_pipeline.query(query, prompt_style=variant)
    )

# Analyze after 1000 queries
results = ab_test.get_experiment_results("prompt_style_test")
print(results)
# {
#   'formal': {'n': 503, 'avg_quality': 0.87, 'avg_user_rating': 4.2},
#   'casual': {'n': 497, 'avg_quality': 0.91, 'avg_user_rating': 4.5},
#   'winner': 'casual'  # Statistically significant
# }
```

Impact: Data-driven prompt optimization; know which changes actually improve quality.

Case 3: Error Replay & Regression Testing

Problem: Production errors are hard to reproduce in development.

Solution: Capture failed traces, replay them as test cases.

python

```
class ErrorReplaySystem:
    def __init__(self, trace_store):
        self.trace_store = trace_store

    def capture_failed_trace(self, trace_id):
        """Capture a failed trace for replay"""
        trace = self.trace_store.get_trace(trace_id)

        # Extract all inputs
        test_case = {
            'trace_id': trace_id,
```

```
            'timestamp': trace.timestamp,
            'inputs': {},
            'expected_behavior': 'should_not_error'
        }

        # Extract inputs from each span
        for span in trace.spans:
            if span.name == "retrieval":
                test_case['inputs']['query'] = span.attributes['query']
                test_case['inputs']['documents'] = span.attributes['documents']

            elif span.name == "generation":
                test_case['inputs']['prompt'] = span.attributes['prompt']
                test_case['inputs']['model'] = span.attributes['model']

            # Capture error
            if span.status.status_code == StatusCode.ERROR:
                test_case['error'] = {
                    'type': span.status.description,
                    'span': span.name,
                    'exception': span.events[0].attributes if span.events else None
                }

        # Save as test case
        self.save_test_case(test_case)

        return test_case

    def replay_trace(self, test_case):
        """Replay captured trace to verify fix"""
        # Reconstruct execution
        try:
            if 'query' in test_case['inputs']:
                retrieval_result = retriever.search(test_case['inputs']['query'])

            if 'prompt' in test_case['inputs']:
                generation_result = llm.generate(
                    test_case['inputs']['prompt'],
                    model=test_case['inputs']['model']
                )

            # Success if no exception
            return {
                'passed': True,
                'error': None
            }
```

```
        except Exception as e:
            # Still failing
            return {
                'passed': False,
                'error': str(e),
                'matches_original': str(e) == test_case['error']['type']
            }

    def create_regression_suite(self):
        """Convert all captured failures into regression tests"""
        failed_traces = self.trace_store.query(
            filters={"status": "error"},
            limit=100
        )

        test_suite = []
        for trace in failed_traces:
            test_case = self.capture_failed_trace(trace.trace_id)
            test_suite.append(test_case)

        # Save as pytest tests
        self.generate_pytest_file(test_suite)

    def generate_pytest_file(self, test_suite):
        """Generate pytest file from captured traces"""
        test_code = """
import pytest
from replay_system import ErrorReplaySystem

replay = ErrorReplaySystem(trace_store)

"""

        for i, test_case in enumerate(test_suite):
            test_code += f"""
def test_regression_{i}_trace_{test_case['trace_id']}():
    '''Regression test from production failure on {test_case['timestamp']}'''
    test_case = {test_case}
    result = replay.replay_trace(test_case)
    assert result['passed'], f"Regression: {{result['error']}}"

"""

        with open('tests/test_regressions.py', 'w') as f:
            f.write(test_code)
```

```
# Usage
replay_system = ErrorReplaySystem(trace_store)

# When error occurs in production
error_trace_id = "abc123"
test_case = replay_system.capture_failed_trace(error_trace_id)

# Developer fixes the issue

# Verify fix
result = replay_system.replay_trace(test_case)
if result['passed']:
    print("Fix verified!")
else:
    print(f"Still failing: {result['error']}")

# Add to regression suite
replay_system.create_regression_suite()
```

Impact: 10x faster debugging; automatic regression prevention.

Expert-Level Comprehension Tests

Question 1: Production users report slow responses. Your trace shows: retrieval=50ms, generation=2500ms, total=2600ms. The generation span has no errors. How do you debug this?

Expected Answer
Step 1: Verify latency is abnormal python

```
# Check if this is an outlier or systemic issue
recent_traces = query_traces(time_range="last_1_hour")

generation_latencies = [
    span.duration_ms
    for trace in recent_traces
    for span in trace.spans
    if span.name == "generation"
]

print(f"P50: {np.percentile(generation_latencies, 50):.0f}ms")
print(f"P95: {np.percentile(generation_latencies, 95):.0f}ms")
print(f"P99: {np.percentile(generation_latencies, 99):.0f}ms")
```

```
# If this trace is p99+, it's an outlier
# If p95 is 2500ms, it's systemic
```

Step 2: Check generation span attributes python

```
gen_span = trace.get_span("generation")

# Token count (most likely culprit)
input_tokens = gen_span.attributes.get('input_tokens')
output_tokens = gen_span.attributes.get('output_tokens')

# Generation rate
tokens_per_second = output_tokens / (gen_span.duration_ms / 1000)

print(f"Input: {input_tokens} tokens")
print(f"Output: {output_tokens} tokens")
print(f"Generation rate: {tokens_per_second:.1f} tok/s")

# GPT-4 typical: 40-60 tok/s
# GPT-3.5 typical: 60-100 tok/s

if output_tokens > 1000:
    print("Issue: Generating too many tokens")
    print("Fix: Add max_tokens limit or stop sequences")

if tokens_per_second < 20:
    print("Issue: Slow generation rate (API issue or model problem)")
    print("Check: API status, model selection")
```

Step 3: Inspect actual prompt python

```
prompt = gen_span.attributes.get('prompt')

# Check for issues:
# 1. Overly long context
if len(prompt.split()) > 3000:
    print("Issue: Context too long")
    print("Fix: Reduce top-k in retrieval or compress context")

# 2. Repetitive content
if has_repetitive_content(prompt):
    print("Issue: Duplicate chunks in context")
    print("Fix: Deduplicate retrieved documents")
```

```
# 3. No max_tokens set
if gen_span.attributes.get('max_tokens') is None:
    print("Issue: No output length limit")
    print("Fix: Set appropriate max_tokens")
```

Step 4: Root cause classification python

```
if output_tokens > 1500:
    root_cause = "EXCESSIVE_OUTPUT"
    fix = "Set max_tokens=500, add stop sequences"

elif input_tokens > 6000:
    root_cause = "EXCESSIVE_INPUT"
    fix = "Reduce retrieval top-k from 10 to 5"

elif tokens_per_second < 20:
    root_cause = "SLOW_GENERATION_RATE"
    fix = "Check API status, consider model switch"

else:
    root_cause = "UNKNOWN"
    fix = "Escalate to OpenAI support with trace_id"
```

Anti-pattern: "Must be the LLM being slow" (no systematic diagnosis)

Question 2: You notice your hallucination rate jumped from 5% to 25% overnight. No code changes were deployed. Using traces, how do you diagnose and fix this?

Expected Answer
Step 1: Identify when degradation started python

```
# Query hallucination metrics over time
metrics = query_metrics(
    metric="hallucination_detected",
    time_range="last_7_days",
    group_by="hour"
)

# Plot
import matplotlib.pyplot as plt
plt.plot(metrics.timestamps, metrics.hallucination_rate)
plt.axhline(y=0.05, color='g', linestyle='--', label='Baseline')
plt.axhline(y=0.25, color='r', linestyle='--', label='Current')
```

```
plt.show()

# Identify exact time of degradation
degradation_start = "2025-02-09 03:00 UTC"
```

Step 2: Compare traces before/after degradation python

```
# Sample traces before and after
before_traces = query_traces(
    filters={"hallucination_detected": True},
    time_range=("2025-02-08 00:00", "2025-02-08 23:59"),
    limit=50
)

after_traces = query_traces(
    filters={"hallucination_detected": True},
    time_range=("2025-02-09 03:00", "2025-02-09 23:59"),
    limit=50
)

# Compare retrieval quality
def analyze_retrieval(traces):
    return {
        'avg_top_score': np.mean([
            t.get_span('retrieval').attributes['top_score']
            for t in traces
        ]),
        'avg_num_docs': np.mean([
            t.get_span('retrieval').attributes['num_docs']
            for t in traces
        ])
    }

before_retrieval = analyze_retrieval(before_traces)
after_retrieval = analyze_retrieval(after_traces)

print("Before:", before_retrieval)
# {'avg_top_score': 0.82, 'avg_num_docs': 5}

print("After:", after_retrieval)
# {'avg_top_score': 0.61, 'avg_num_docs': 5}

# Smoking gun: Retrieval quality dropped!
```

Step 3: Investigate why retrieval quality dropped python

```
# Hypothesis 1: Embedding model changed
# Check model versions
before_model = before_traces[0].get_span('retrieval').attributes.get('embedding_model')
after_model = after_traces[0].get_span('retrieval').attributes.get('embedding_model')

if before_model != after_model:
    print(f"Model changed: {before_model} → {after_model}")
    print("Fix: Rollback embedding model or re-index")

# Hypothesis 2: Index corruption
# Check index health
index_stats = vector_db.get_stats()
if index_stats['last_modified'] > degradation_start:
    print("Index was modified during degradation window")
    print("Fix: Restore index from backup")

# Hypothesis 3: Documents removed
if index_stats['doc_count'] < expected_doc_count:
    print(f"Missing documents: {expected_doc_count - index_stats['doc_count']}")
    print("Fix: Re-index missing documents")

# Hypothesis 4: Query pattern shift
# Check if query distribution changed
before_queries = [t.get_span('retrieval').attributes['query'] for t in before_traces]
after_queries = [t.get_span('retrieval').attributes['query'] for t in after_traces]

# Cluster queries
from sklearn.cluster import KMeans
before_clusters = cluster_queries(before_queries)
after_clusters = cluster_queries(after_queries)

if before_clusters != after_clusters:
    print("Query distribution shifted")
    print("Fix: This is user behavior change, not system issue")
```

Step 4: Verify fix python

```
# After implementing fix, compare metrics
fixed_traces = query_traces(
    time_range="last_1_hour",
    limit=100
)
```

```
fixed_hallucination_rate = sum(
    1 for t in fixed_traces
    if t.attributes.get('hallucination_detected')
) / len(fixed_traces)

print(f"Hallucination rate after fix: {fixed_hallucination_rate:.1%}")
# Target: <10%
```

Root causes by likelihood: 1. Vector database index changed/corrupted (40%) 2. Embedding model version updated (25%) 3. Documents removed from index (15%) 4. Query pattern shifted (15%) 5. Other (5%)

Question 3: Your observability system is logging 1M traces/day at $0.001/trace = $1000/day. How do you reduce costs by 90% while maintaining debuggability?

Expected Answer
Strategy 1: Intelligent Sampling python

```
class SmartSampler:
    def __init__(self, base_rate=0.1):
        self.base_rate = base_rate  # Sample 10% by default

    def should_trace(self, request):
        """Decide whether to create full trace"""

        # Always trace:
        # 1. Errors
        if request.has_error:
            return True

        # 2. Slow requests (p95+)
        if request.latency_ms > self.get_p95_latency():
            return True

        # 3. Low quality (hallucinations, low scores)
        if request.quality_score < 0.7:
            return True

        # 4. First query per session (debugging context)
        if request.is_first_in_session:
            return True

        # 5. Flagged users (VIP customers, testers)
        if request.user_id in self.flagged_users:
```

```
            return True

        # 6. Sample of normal traffic (base rate)
        return random.random() < self.base_rate

# Result: Trace 30-40% of requests instead of 100%
# Cost reduction: 60-70%
```

Strategy 2: Tiered Storage python

```
class TieredTraceStorage:
    def __init__(self):
        self.hot_storage = PostgreSQL()  # Fast, expensive
        self.cold_storage = S3()         # Slow, cheap

    def store_trace(self, trace):
        # Compute trace value score
        value = self.calculate_value(trace)

        if value > 0.8:  # High value
            # Store in hot storage (queryable)
            self.hot_storage.insert(trace)
            ttl = 30  # days

        elif value > 0.5:  # Medium value
            # Store summary in hot, full trace in cold
            self.hot_storage.insert(trace.summary())
            self.cold_storage.upload(trace)
            ttl = 7  # days in hot

        else:  # Low value
            # Store aggregated metrics only
            self.store_aggregated_metrics(trace)
            # Don't store full trace

        # Set TTL for automatic deletion
        self.hot_storage.set_ttl(trace.trace_id, ttl)

    def calculate_value(self, trace):
        """How valuable is this trace for debugging?"""
        score = 0

        # Errors are very valuable
        if trace.has_error:
            score += 0.5
```

```
        # Outliers are valuable
        if trace.is_latency_outlier or trace.is_quality_outlier:
            score += 0.3

        # User feedback is valuable
        if trace.has_user_feedback:
            score += 0.3

        # A/B test traces are valuable
        if trace.is_ab_test:
            score += 0.2

        return min(score, 1.0)

# Result: Store 20% as full traces, 30% as summaries, 50% as aggregates
# Storage cost reduction: 70%
```

Strategy 3: Compression & Aggregation python

```
class CompressedTraceStorage:
    def store_trace(self, trace):
        # Instead of storing full prompt/response
        compressed = {
            'trace_id': trace.trace_id,
            'timestamp': trace.timestamp,
            'user_id': trace.user_id,

            # Store hashes instead of full text (90% size reduction)
            'prompt_hash': hash(trace.prompt),
            'response_hash': hash(trace.response),

            # Store only metrics
            'latency_ms': trace.latency_ms,
            'tokens_in': trace.tokens_in,
            'tokens_out': trace.tokens_out,
            'cost': trace.cost,
            'quality_score': trace.quality_score,

            # Store only on errors
            'full_trace': trace.to_json() if trace.has_error else None
        }

        self.db.insert(compressed)
```

```
# For debugging, retrieve full trace only when needed
def debug_trace(trace_id):
    compressed = db.get(trace_id)

    if compressed['full_trace']:
        return compressed['full_trace']
    else:
        # Reconstruct from metrics (limited info)
        return reconstruct_trace(compressed)
```

Strategy 4: Sampling Patterns python

```
# Head sampling: Decide before execution
def head_sample():
    if random.random() < 0.1:  # 10% sample rate
        return create_trace()
    else:
        return no_trace()

# Tail sampling: Decide after execution (smarter)
def tail_sample(trace):
    # See results before deciding to keep
    if trace.has_error or trace.latency > p95:
        store_trace(trace)
    else:
        discard_trace(trace)

# Adaptive sampling: Adjust rate dynamically
def adaptive_sample():
    current_error_rate = get_current_error_rate()

    if current_error_rate > 0.1:  # High errors
        return 0.5  # Sample 50%
    elif current_error_rate > 0.05:
        return 0.2  # Sample 20%
    else:
        return 0.05  # Sample 5%
```

Combined Strategy: python

```
class CostOptimizedObservability:
    def __init__(self):
        self.sampler = SmartSampler(base_rate=0.05)  # 5% base
        self.storage = TieredTraceStorage()
```

```
    def track_request(self, request):
        # Decide if should trace
        if not self.sampler.should_trace(request):
            # Still collect lightweight metrics
            self.store_metrics_only(request)
            return

        # Create trace
        with tracer.start_as_current_span("request") as span:
            # ... execute request ...

            # Tail sampling: Keep or discard?
            if self.should_keep_trace(span):
                self.storage.store_trace(span)
            # else: trace discarded after collection

# Results:
# Before: 1M traces/day × $0.001 = $1000/day
# After:
#   - 50k full traces (5% sample) × $0.001 = $50/day
#   - 200k summaries × $0.0001 = $20/day
#   - 750k metrics only × $0.00001 = $7.50/day
# Total: $77.50/day (92% reduction)
```

Key principle: **Not all traces are equally valuable. Store the valuable ones.**

---

3. Fastest Path to Competence (Execution-Focused)

Prerequisites (Week 0: Setup)

Install:

bash

```
pip install \
    opentelemetry-api \
    opentelemetry-sdk \
    opentelemetry-instrumentation \
    langfuse \              # LLM-specific observability
    phoenix-ai \            # Alternative: Arize Phoenix
```

```
    langsmith \             # Alternative: LangSmith
    psycopg2 \              # For trace storage
    redis \                 # For caching
    prometheus-client       # Metrics
```

Repository structure:

```
llm-observability/
├── tracing/
│   ├── instrumentation.py    # OpenTelemetry setup
│   ├── custom_spans.py        # LLM-specific spans
│   └── samplers.py            # Sampling strategies
├── storage/
│   ├── trace_db.py            # Trace persistence
│   └── queries.py             # Trace querying
├── monitoring/
│   ├── dashboards.py          # Metrics dashboards
│   └── alerts.py              # Alerting rules
├── debugging/
│   ├── trace_viewer.py        # Trace visualization
│   └── error_replay.py        # Reproduce errors
└── README.md
```

Week 1: Basic Tracing & Instrumentation

Learning Objectives:

- Set up OpenTelemetry tracing
- Instrument LLM calls with spans
- Store and query traces
- Build trace visualization

Day 1-2: OpenTelemetry Setup

Hands-on Lab:

python

```
# tracing/instrumentation.py
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
```

```
from opentelemetry.sdk.trace.export import BatchSpanProcessor, ConsoleSpanExporter
from opentelemetry.sdk.resources import Resource
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter

# Configure tracer provider
resource = Resource.create({
    "service.name": "llm-application",
    "service.version": "1.0.0",
    "deployment.environment": "production"
})

provider = TracerProvider(resource=resource)

# Console exporter for development
console_exporter = ConsoleSpanExporter()
provider.add_span_processor(BatchSpanProcessor(console_exporter))

# OTLP exporter for production (sends to collector)
otlp_exporter = OTLPSpanExporter(
    endpoint="http://localhost:4317",  # OpenTelemetry Collector
    insecure=True
)
provider.add_span_processor(BatchSpanProcessor(otlp_exporter))

# Set global tracer provider
trace.set_tracer_provider(provider)

# Get tracer for this module
tracer = trace.get_tracer(__name__)

# Basic usage
def simple_llm_call(prompt):
    with tracer.start_as_current_span("llm_generation") as span:
        # Add attributes
        span.set_attribute("prompt", prompt)
        span.set_attribute("model", "gpt-3.5-turbo")

        # Call LLM
        response = openai.ChatCompletion.create(
            model="gpt-3.5-turbo",
            messages=[{"role": "user", "content": prompt}]
        )

        # Add response attributes
        span.set_attribute("response", response.choices[0].message.content)
        span.set_attribute("tokens_in", response.usage.prompt_tokens)
```

```
        span.set_attribute("tokens_out", response.usage.completion_tokens)

        return response.choices[0].message.content

# Test
result = simple_llm_call("What is Python?")
print(result)
```

Deliverable: Working tracer that:

- Exports to console (for debugging)
- Exports to OTLP collector (for production)
- Captures basic span attributes

Resources:

- OpenTelemetry Python Docs
- OTLP Specification

Day 3-4: Multi-Span RAG Pipeline

Hands-on Lab:

python

```
# tracing/custom_spans.py
from opentelemetry import trace
from opentelemetry.trace import Status, StatusCode
import time

tracer = trace.get_tracer(__name__)

class TracedRAGPipeline:
    def __init__(self, retriever, llm_client):
        self.retriever = retriever
        self.llm = llm_client

    def query(self, question, user_id=None, session_id=None):
        # Root span
        with tracer.start_as_current_span("rag_query") as root_span:
            # Metadata
            root_span.set_attribute("question", question)
            if user_id:
                root_span.set_attribute("user_id", user_id)
            if session_id:
```

```
                root_span.set_attribute("session_id", session_id)

            try:
                # Retrieval span (child of root)
                docs = self._traced_retrieval(question)

                # Generation span (child of root)
                answer = self._traced_generation(question, docs)

                # Post-processing span
                final_answer = self._traced_postprocess(answer)

                # Success
                root_span.set_status(Status(StatusCode.OK))
                root_span.set_attribute("success", True)

                return final_answer

            except Exception as e:
                # Error
                root_span.set_status(Status(StatusCode.ERROR, str(e)))
                root_span.record_exception(e)
                root_span.set_attribute("success", False)
                raise

    def _traced_retrieval(self, question):
        with tracer.start_as_current_span("retrieval") as span:
            start_time = time.time()

            # Retrieve
            docs = self.retriever.search(question, top_k=5)

            # Metrics
            span.set_attribute("num_docs_retrieved", len(docs))
            span.set_attribute("top_score", docs[0]['score'] if docs else 0)
            span.set_attribute("retrieval_latency_ms",
                               (time.time() - start_time) * 1000)

            # Add event for each retrieved doc
            for i, doc in enumerate(docs):
                span.add_event(f"doc_{i}_retrieved", {
                    "score": doc['score'],
                    "source": doc.get('metadata', {}).get('source', 'unknown'
                })

            return docs
```

```
    def _traced_generation(self, question, docs):
        with tracer.start_as_current_span("generation") as span:
            # Build prompt
            context = "\n\n".join([d['text'] for d in docs])
            prompt = f"Context:\n{context}\n\nQuestion: {question}\nAnswer:"

            # Attributes
            span.set_attribute("prompt_template", "default_rag")
            span.set_attribute("num_context_docs", len(docs))

            start_time = time.time()

            # Generate
            response = self.llm.chat.completions.create(
                model="gpt-3.5-turbo",
                messages=[{"role": "user", "content": prompt}],
                temperature=0
            )

            answer = response.choices[0].message.content

            # Metrics
            span.set_attribute("model", "gpt-3.5-turbo")
            span.set_attribute("temperature", 0)
            span.set_attribute("tokens_in", response.usage.prompt_tokens)
            span.set_attribute("tokens_out", response.usage.completion_tokens
            span.set_attribute("generation_latency_ms",
                               (time.time() - start_time) * 1000)

            # Cost calculation
            cost = (
                response.usage.prompt_tokens * 0.0015 / 1000 +
                response.usage.completion_tokens * 0.002 / 1000
            )
            span.set_attribute("cost_usd", cost)

            return answer

    def _traced_postprocess(self, answer):
        with tracer.start_as_current_span("postprocess") as span:
            # Add citations, formatting, etc.
            processed = f"Answer: {answer}\n\n[Generated by AI]"

            span.set_attribute("processing_applied", "add_disclaimer")
```

```
            return processed

# Usage
from openai import OpenAI

llm = OpenAI()
rag = TracedRAGPipeline(retriever, llm)

answer = rag.query(
    "How do I reset my password?",
    user_id="user_123",
    session_id="session_abc"
)
```

Trace visualization:

```
Trace: rag_query (1.05s)
├─ retrieval (0.05s)
│  ├─ num_docs_retrieved: 5
│  ├─ top_score: 0.87
│  └─ events:
│     ├─ doc_0_retrieved (score: 0.87, source: "manual.pdf")
│     └─ doc_1_retrieved (score: 0.82, source: "faq.md")
├─ generation (0.95s)
│  ├─ model: "gpt-3.5-turbo"
│  ├─ tokens_in: 1200
│  ├─ tokens_out: 150
│  └─ cost_usd: 0.0021
└─ postprocess (0.05s)
   └─ processing_applied: "add_disclaimer"
```

Deliverable: Multi-span pipeline showing:

- Parent-child relationships
- Latency breakdown
- Token usage
- Cost tracking

Day 5-7: Trace Storage & Querying

Hands-on Lab:

python

```
# storage/trace_db.py
import psycopg2
from psycopg2.extras import Json
from datetime import datetime
from typing import List, Dict, Any

class TraceDatabase:
    def __init__(self, connection_string):
        self.conn = psycopg2.connect(connection_string)
        self.create_tables()

    def create_tables(self):
        """Create trace storage schema"""
        with self.conn.cursor() as cur:
            cur.execute("""
                CREATE TABLE IF NOT EXISTS traces (
                    trace_id VARCHAR(32) PRIMARY KEY,
                    timestamp TIMESTAMP NOT NULL,
                    user_id VARCHAR(255),
                    session_id VARCHAR(255),
                    status VARCHAR(20),
                    duration_ms INTEGER,
                    attributes JSONB,
                    INDEX idx_timestamp (timestamp),
                    INDEX idx_user_id (user_id),
                    INDEX idx_session (session_id)
                );

                CREATE TABLE IF NOT EXISTS spans (
                    span_id VARCHAR(32) PRIMARY KEY,
                    trace_id VARCHAR(32) REFERENCES traces(trace_id),
                    parent_span_id VARCHAR(32),
                    name VARCHAR(255),
                    start_time TIMESTAMP,
                    end_time TIMESTAMP,
                    duration_ms INTEGER,
                    status VARCHAR(20),
                    attributes JSONB,
                    events JSONB,
                    INDEX idx_trace (trace_id),
                    INDEX idx_name (name)
                );
            """)
            self.conn.commit()
```

```
    def store_trace(self, trace):
        """Store complete trace"""
        with self.conn.cursor() as cur:
            # Store trace
            cur.execute("""
                INSERT INTO traces (
                    trace_id, timestamp, user_id, session_id,
                    status, duration_ms, attributes
                ) VALUES (%s, %s, %s, %s, %s, %s, %s)
                ON CONFLICT (trace_id) DO UPDATE SET
                    status = EXCLUDED.status,
                    duration_ms = EXCLUDED.duration_ms,
                    attributes = EXCLUDED.attributes
            """, (
                trace.trace_id,
                datetime.fromtimestamp(trace.start_time / 1e9),
                trace.attributes.get('user_id'),
                trace.attributes.get('session_id'),
                trace.status,
                trace.duration_ms,
                Json(dict(trace.attributes))
            ))

            # Store spans
            for span in trace.spans:
                cur.execute("""
                    INSERT INTO spans (
                        span_id, trace_id, parent_span_id, name,
                        start_time, end_time, duration_ms,
                        status, attributes, events
                    ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
                    ON CONFLICT (span_id) DO UPDATE SET
                        status = EXCLUDED.status,
                        attributes = EXCLUDED.attributes
                """, (
                    span.span_id,
                    trace.trace_id,
                    span.parent_span_id,
                    span.name,
                    datetime.fromtimestamp(span.start_time / 1e9),
                    datetime.fromtimestamp(span.end_time / 1e9),
                    span.duration_ms,
                    span.status,
                    Json(dict(span.attributes)),
                    Json([e.to_dict() for e in span.events])
                ))
```

```
            self.conn.commit()

    def query_traces(self,
                     user_id: str = None,
                     session_id: str = None,
                     status: str = None,
                     time_range: tuple = None,
                     limit: int = 100) -> List[Dict]:
        """Query traces with filters"""

        query = "SELECT * FROM traces WHERE 1=1"
        params = []

        if user_id:
            query += " AND user_id = %s"
            params.append(user_id)

        if session_id:
            query += " AND session_id = %s"
            params.append(session_id)

        if status:
            query += " AND status = %s"
            params.append(status)

        if time_range:
            query += " AND timestamp BETWEEN %s AND %s"
            params.extend(time_range)

        query += " ORDER BY timestamp DESC LIMIT %s"
        params.append(limit)

        with self.conn.cursor() as cur:
            cur.execute(query, params)
            columns = [desc[0] for desc in cur.description]
            return [dict(zip(columns, row)) for row in cur.fetchall()]

    def get_trace_with_spans(self, trace_id: str) -> Dict:
        """Get complete trace with all spans"""
        with self.conn.cursor() as cur:
            # Get trace
            cur.execute("SELECT * FROM traces WHERE trace_id = %s", (trace_id,))
            trace = dict(zip([d[0] for d in cur.description], cur.fetchone()))

            # Get spans
```

```
            cur.execute("""
                SELECT * FROM spans
                WHERE trace_id = %s
                ORDER BY start_time
            """, (trace_id,))

            columns = [d[0] for d in cur.description]
            trace['spans'] = [dict(zip(columns, row)) for row in cur.fetchall()]

            return trace

    def get_slow_traces(self, threshold_ms: int = 2000, limit: int = 10):
        """Find slowest traces"""
        with self.conn.cursor() as cur:
            cur.execute("""
                SELECT trace_id, timestamp, duration_ms, attributes
                FROM traces
                WHERE duration_ms > %s
                ORDER BY duration_ms DESC
                LIMIT %s
            """, (threshold_ms, limit))

            columns = [d[0] for d in cur.description]
            return [dict(zip(columns, row)) for row in cur.fetchall()]

    def get_error_traces(self, time_range: tuple = None, limit: int = 100):
        """Find error traces"""
        query = "SELECT * FROM traces WHERE status = 'ERROR'"
        params = []

        if time_range:
            query += " AND timestamp BETWEEN %s AND %s"
            params.extend(time_range)

        query += " ORDER BY timestamp DESC LIMIT %s"
        params.append(limit)

        with self.conn.cursor() as cur:
            cur.execute(query, params)
            columns = [d[0] for d in cur.description]
            return [dict(zip(columns, row)) for row in cur.fetchall()]

# Usage
db = TraceDatabase("postgresql://localhost/traces")

# Query examples
```

```
# 1. All traces for a user
user_traces = db.query_traces(user_id="user_123", limit=50)

# 2. Error traces in last hour
from datetime import datetime, timedelta
one_hour_ago = datetime.now() - timedelta(hours=1)
errors = db.get_error_traces(
    time_range=(one_hour_ago, datetime.now())
)

# 3. Slow traces
slow_traces = db.get_slow_traces(threshold_ms=2000)

# 4. Complete trace with spans
full_trace = db.get_trace_with_spans("abc123")
print(f"Trace duration: {full_trace['duration_ms']}ms")
for span in full_trace['spans']:
    print(f"  {span['name']}: {span['duration_ms']}ms")
```

Deliverable: Queryable trace database with:

- Efficient indexes
- Flexible filtering
- Full trace reconstruction

Week 2: Advanced Monitoring & Alerting

Learning Objectives:

- Build metrics dashboards
- Implement alerting rules
- Create error classification
- Set up automated debugging

Day 8-10: Metrics & Dashboards

Hands-on Lab:

python

```
# monitoring/dashboards.py
from prometheus_client import Counter, Histogram, Gauge, start_http_server
```

```
from dataclasses import dataclass
from typing import Dict
import time

# Define metrics
llm_requests_total = Counter(
    'llm_requests_total',
    'Total LLM requests',
    ['model', 'status']
)

llm_request_duration = Histogram(
    'llm_request_duration_seconds',
    'LLM request duration',
    ['model', 'operation'],
    buckets=[0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
)

llm_tokens = Counter(
    'llm_tokens_total',
    'Total tokens processed',
    ['model', 'direction']  # direction: input/output
)

llm_cost = Counter(
    'llm_cost_usd_total',
    'Total cost in USD',
    ['model']
)

llm_quality_score = Histogram(
    'llm_quality_score',
    'Quality score distribution',
    ['metric_name'],
    buckets=[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
)

active_sessions = Gauge(
    'llm_active_sessions',
    'Number of active sessions'
)

class MetricsCollector:
    def __init__(self):
        self.active_sessions_set = set()
```

```
    def record_request(self,
                       model: str,
                       operation: str,
                       duration: float,
                       tokens_in: int,
                       tokens_out: int,
                       cost: float,
                       status: str = "success",
                       quality_scores: Dict[str, float] = None):
        """Record all metrics for a request"""

        # Count
        llm_requests_total.labels(model=model, status=status).inc()

        # Duration
        llm_request_duration.labels(model=model, operation=operation).observe(duration)

        # Tokens
        llm_tokens.labels(model=model, direction="input").inc(tokens_in)
        llm_tokens.labels(model=model, direction="output").inc(tokens_out)

        # Cost
        llm_cost.labels(model=model).inc(cost)

        # Quality
        if quality_scores:
            for metric_name, score in quality_scores.items():
                llm_quality_score.labels(metric_name=metric_name).observe(score)

    def track_session(self, session_id: str, active: bool):
        """Track active sessions"""
        if active:
            self.active_sessions_set.add(session_id)
        else:
            self.active_sessions_set.discard(session_id)

        active_sessions.set(len(self.active_sessions_set))

# Instrumented RAG pipeline
class MonitoredRAGPipeline:
    def __init__(self, rag_pipeline, metrics_collector):
        self.rag = rag_pipeline
        self.metrics = metrics_collector

    def query(self, question, session_id=None):
        self.metrics.track_session(session_id, active=True)
```

```
        start = time.time()

        try:
            # Execute
            result = self.rag.query(question)
            duration = time.time() - start

            # Record metrics
            self.metrics.record_request(
                model="gpt-3.5-turbo",
                operation="rag_query",
                duration=duration,
                tokens_in=result['tokens_in'],
                tokens_out=result['tokens_out'],
                cost=result['cost'],
                status="success",
                quality_scores={
                    'relevance': result['relevance_score'],
                    'groundedness': result['groundedness_score']
                }
            )

            return result

        except Exception as e:
            duration = time.time() - start

            self.metrics.record_request(
                model="gpt-3.5-turbo",
                operation="rag_query",
                duration=duration,
                tokens_in=0,
                tokens_out=0,
                cost=0,
                status="error"
            )

            raise

        finally:
            self.metrics.track_session(session_id, active=False)

# Start metrics server
start_http_server(8000)  # Metrics available at http://localhost:8000/metrics
```

```
# Usage
metrics = MetricsCollector()
monitored_rag = MonitoredRAGPipeline(rag_pipeline, metrics)

# Metrics exported in Prometheus format:
# llm_requests_total{model="gpt-3.5-turbo",status="success"} 1523
# llm_request_duration_seconds_bucket{model="gpt-3.5-turbo",operation="rag_query",le="1.
# llm_tokens_total{model="gpt-3.5-turbo",direction="input"} 1500000
# llm_cost_usd_total{model="gpt-3.5-turbo"} 45.67
```

Grafana Dashboard Configuration:

yaml

```
# dashboards/llm_overview.json
{
  "dashboard": {
    "title": "LLM Application Overview",
    "panels": [
      {
        "title": "Request Rate",
        "targets": [{
          "expr": "rate(llm_requests_total[5m])"
        }]
      },
      {
        "title": "P95 Latency",
        "targets": [{
          "expr": "histogram_quantile(0.95, llm_request_duration_seconds_bucket)"
        }]
      },
      {
        "title": "Error Rate",
        "targets": [{
          "expr": "rate(llm_requests_total{status=\"error\"}[5m]) / rate(llm_requests_to
        }]
      },
      {
        "title": "Cost per Hour",
        "targets": [{
          "expr": "increase(llm_cost_usd_total[1h])"
        }]
      },
      {
        "title": "Quality Scores",
```

```
        "targets": [
          {"expr": "llm_quality_score{metric_name=\"relevance\"}"},
          {"expr": "llm_quality_score{metric_name=\"groundedness\"}"}
        ]
      }
    ]
  }
}
```

Deliverable: Monitoring system with:

- Prometheus metrics export
- Grafana dashboards
- Real-time quality tracking

Day 11-13: Alerting & Incident Response

Hands-on Lab:

python

```
# monitoring/alerts.py
from dataclasses import dataclass
from typing import Callable, Dict, Any
from enum import Enum
import logging

class Severity(Enum):
    INFO = "info"
    WARNING = "warning"
    ERROR = "error"
    CRITICAL = "critical"

@dataclass
class Alert:
    name: str
    severity: Severity
    message: str
    details: Dict[str, Any]
    timestamp: float

class AlertManager:
    def __init__(self, trace_db, metrics_collector):
        self.trace_db = trace_db
        self.metrics = metrics_collector
```

```
        self.alert_handlers = []

        # Alert rules
        self.rules = {
            'high_error_rate': {
                'condition': lambda: self.get_error_rate() > 0.1,
                'severity': Severity.CRITICAL,
                'message': 'Error rate exceeds 10%',
                'playbook': self.high_error_rate_playbook
            },
            'high_latency': {
                'condition': lambda: self.get_p95_latency() > 3000,
                'severity': Severity.WARNING,
                'message': 'P95 latency exceeds 3s',
                'playbook': self.high_latency_playbook
            },
            'quality_degradation': {
                'condition': lambda: self.get_avg_quality() < 0.7,
                'severity': Severity.ERROR,
                'message': 'Average quality score below 0.7',
                'playbook': self.quality_degradation_playbook
            },
            'cost_spike': {
                'condition': lambda: self.get_hourly_cost() > 50,
                'severity': Severity.WARNING,
                'message': 'Hourly cost exceeds $50',
                'playbook': self.cost_spike_playbook
            }
        }

    def check_alerts(self):
        """Check all alert rules"""
        for rule_name, rule in self.rules.items():
            if rule['condition']():
                alert = Alert(
                    name=rule_name,
                    severity=rule['severity'],
                    message=rule['message'],
                    details=self.get_rule_details(rule_name),
                    timestamp=time.time()
                )

                self.fire_alert(alert)

                # Run playbook
                if rule.get('playbook'):
```

```
                    rule['playbook'](alert)

    def fire_alert(self, alert: Alert):
        """Send alert to all handlers"""
        for handler in self.alert_handlers:
            handler(alert)

    def add_alert_handler(self, handler: Callable):
        """Add alert notification handler"""
        self.alert_handlers.append(handler)

    # Playbooks (automated responses)

    def high_error_rate_playbook(self, alert: Alert):
        """Automated response to high error rate"""
        print(f"[PLAYBOOK] Responding to: {alert.name}")

        # 1. Get recent error traces
        errors = self.trace_db.get_error_traces(limit=20)

        # 2. Classify errors
        error_types = self.classify_errors(errors)

        # 3. Take action based on error type
        if error_types.get('rate_limit', 0) > 0.5:
            print("  → Most errors are rate limits")
            print("  → Action: Enable request queuing")
            # self.enable_request_queue()

        elif error_types.get('timeout', 0) > 0.5:
            print("  → Most errors are timeouts")
            print("  → Action: Increase timeout threshold")
            # self.increase_timeout()

        else:
            print("  → Mixed error types")
            print("  → Action: Page on-call engineer")
            # self.page_oncall()

    def high_latency_playbook(self, alert: Alert):
        """Automated response to high latency"""
        print(f"[PLAYBOOK] Responding to: {alert.name}")

        # 1. Get slow traces
        slow_traces = self.trace_db.get_slow_traces(threshold_ms=3000, limit=10)
```

```
        # 2. Analyze latency breakdown
        for trace in slow_traces[:3]:
            full_trace = self.trace_db.get_trace_with_spans(trace['trace_id'])

            print(f"  Trace {trace['trace_id']}:")
            for span in full_trace['spans']:
                pct = span['duration_ms'] / trace['duration_ms'] * 100
                print(f"    {span['name']}: {span['duration_ms']}ms ({pct:.1f}%)")

        # 3. Identify bottleneck
        # If >80% time in one span, that's the bottleneck

    def quality_degradation_playbook(self, alert: Alert):
        """Automated response to quality issues"""
        print(f"[PLAYBOOK] Responding to: {alert.name}")

        # 1. Sample recent traces
        recent = self.trace_db.query_traces(limit=100)

        # 2. Check retrieval quality
        retrieval_scores = []
        for trace in recent:
            full = self.trace_db.get_trace_with_spans(trace['trace_id'])
            retrieval_span = next(
                (s for s in full['spans'] if s['name'] == 'retrieval'),
                None
            )
            if retrieval_span:
                retrieval_scores.append(
                    retrieval_span['attributes'].get('top_score', 0)
                )

        avg_retrieval = np.mean(retrieval_scores) if retrieval_scores else 0

        if avg_retrieval < 0.5:
            print("  → Root cause: Poor retrieval quality")
            print("  → Action: Check vector DB index health")
        else:
            print("  → Root cause: Generation quality issue")
            print("  → Action: Review prompt templates")

    def cost_spike_playbook(self, alert: Alert):
        """Automated response to cost spike"""
        print(f"[PLAYBOOK] Responding to: {alert.name}")

        # Analyze cost breakdown
```

```
        # Enable rate limiting if necessary

# Alert handlers

def slack_handler(alert: Alert):
    """Send to Slack"""
    import requests

    color = {
        Severity.INFO: "good",
        Severity.WARNING: "warning",
        Severity.ERROR: "danger",
        Severity.CRITICAL: "danger"
    }[alert.severity]

    requests.post(
        "https://hooks.slack.com/services/YOUR/WEBHOOK/URL",
        json={
            "attachments": [{
                "color": color,
                "title": f"[{alert.severity.value.upper()}] {alert.name}",
                "text": alert.message,
                "fields": [
                    {"title": k, "value": str(v), "short": True}
                    for k, v in alert.details.items()
                ]
            }]
        }
    )

def pagerduty_handler(alert: Alert):
    """Page on-call for critical alerts"""
    if alert.severity == Severity.CRITICAL:
        # Trigger PagerDuty incident
        pass

# Usage
alert_mgr = AlertManager(trace_db, metrics)

# Add handlers
alert_mgr.add_alert_handler(slack_handler)
alert_mgr.add_alert_handler(pagerduty_handler)

# Run alert checks periodically
import schedule
schedule.every(1).minutes.do(alert_mgr.check_alerts)
```

Deliverable: Alerting system with:

- Configurable alert rules
- Multiple notification channels
- Automated playbooks
- Incident classification

Day 14: Error Classification

Hands-on Lab:

python

```
# debugging/error_classifier.py
from enum import Enum
from typing import List, Dict
from collections import Counter

class ErrorCategory(Enum):
    RATE_LIMIT = "rate_limit"
    TIMEOUT = "timeout"
    INVALID_REQUEST = "invalid_request"
    VALIDATION_ERROR = "validation_error"
    HALLUCINATION = "hallucination"
    CONTEXT_OVERFLOW = "context_overflow"
    API_ERROR = "api_error"
    UNKNOWN = "unknown"

class ErrorClassifier:
    def __init__(self):
        # Error patterns
        self.patterns = {
            ErrorCategory.RATE_LIMIT: [
                "rate limit", "429", "too many requests"
            ],
            ErrorCategory.TIMEOUT: [
                "timeout", "timed out", "deadline exceeded"
            ],
            ErrorCategory.INVALID_REQUEST: [
                "invalid request", "bad request", "400"
            ],
            ErrorCategory.VALIDATION_ERROR: [
                "validation error", "schema error", "pydantic"
            ],
            ErrorCategory.CONTEXT_OVERFLOW: [
```

```
                "context length", "token limit", "too long"
            ],
            ErrorCategory.API_ERROR: [
                "500", "502", "503", "internal server error"
            ]
        }

    def classify(self, error_message: str, span_data: Dict = None) -> ErrorCategory:
        """Classify error into category"""
        error_lower = error_message.lower()

        # Pattern matching
        for category, patterns in self.patterns.items():
            if any(p in error_lower for p in patterns):
                return category

        # Span-based classification
        if span_data:
            # Check for hallucination
            if span_data.get('hallucination_detected'):
                return ErrorCategory.HALLUCINATION

        return ErrorCategory.UNKNOWN

    def classify_batch(self, error_traces: List[Dict]) -> Dict[ErrorCategory, List]:
        """Classify multiple errors"""
        classified = {cat: [] for cat in ErrorCategory}

        for trace in error_traces:
            # Get error message from trace
            error_msg = trace['attributes'].get('error_message', '')

            # Get span data
            full_trace = trace_db.get_trace_with_spans(trace['trace_id'])
            error_span = next(
                (s for s in full_trace['spans'] if s['status'] == 'ERROR'),
                None
            )

            # Classify
            category = self.classify(error_msg, error_span['attributes'] if error_span e
            classified[category].append(trace)

        return classified

    def generate_error_report(self, time_range: tuple = None):
```

```
        """Generate error analysis report"""
        # Get errors
        errors = trace_db.get_error_traces(time_range=time_range)

        # Classify
        classified = self.classify_batch(errors)

        # Count
        counts = {cat: len(traces) for cat, traces in classified.items() if traces}

        # Generate report
        report = f"""
ERROR ANALYSIS REPORT
{'='*60}
Time Range: {time_range[0]} to {time_range[1]}
Total Errors: {len(errors)}

Breakdown by Category:
"""
        for category, count in sorted(counts.items(), key=lambda x: x[1], reverse=True):
            pct = count / len(errors) * 100
            report += f"\n{category.value:20s}: {count:4d} ({pct:5.1f}%)"

            # Show example for top categories
            if count >= 3:
                examples = classified[category][:2]
                for ex in examples:
                    report += f"\n  → {ex['trace_id']}: {ex['attributes'].get('error_mes

        # Recommendations
        report += "\n\nRECOMMENDATIONS:\n"

        if counts.get(ErrorCategory.RATE_LIMIT, 0) > len(errors) * 0.3:
            report += "\n- High rate limit errors: Implement request queuing or increase

        if counts.get(ErrorCategory.TIMEOUT, 0) > len(errors) * 0.2:
            report += "\n- Frequent timeouts: Optimize latency or increase timeout thres

        if counts.get(ErrorCategory.CONTEXT_OVERFLOW, 0) > 0:
            report += "\n- Context overflow: Reduce retrieval top-k or compress context"

        return report

# Usage
classifier = ErrorClassifier()
```

```
# Classify single error
error_msg = "Error: OpenAI API rate limit exceeded"
category = classifier.classify(error_msg)
print(f"Category: {category}")  # ErrorCategory.RATE_LIMIT

# Generate report
from datetime import datetime, timedelta
one_day_ago = datetime.now() - timedelta(days=1)
report = classifier.generate_error_report(
    time_range=(one_day_ago, datetime.now())
)
print(report)
```

Deliverable: Error classification system with:

- Automated categorization
- Error pattern detection
- Actionable recommendations

Week 3: Production Observability & Portfolio

Learning Objectives:

- Implement production-grade observability
- Build trace visualization UI
- Create observability best practices doc
- Portfolio demonstration

Day 15-17: Trace Visualization

Hands-on Lab:

python

```
# debugging/trace_viewer.py
import streamlit as st
import pandas as pd
from datetime import datetime, timedelta

class TraceViewer:
    def __init__(self, trace_db):
        self.db = trace_db
```

```
    def render(self):
        st.title("LLM Trace Viewer")

        # Filters
        col1, col2, col3 = st.columns(3)

        with col1:
            user_id = st.text_input("User ID")

        with col2:
            time_range_hours = st.slider("Time Range (hours)", 1, 24, 6)

        with col3:
            status = st.selectbox("Status", ["All", "OK", "ERROR"])

        # Query traces
        start_time = datetime.now() - timedelta(hours=time_range_hours)

        filters = {}
        if user_id:
            filters['user_id'] = user_id
        if status != "All":
            filters['status'] = status

        traces = self.db.query_traces(
            **filters,
            time_range=(start_time, datetime.now()),
            limit=100
        )

        # Display trace list
        st.subheader(f"Found {len(traces)} traces")

        if traces:
            # Create DataFrame
            df = pd.DataFrame([{
                'Trace ID': t['trace_id'][:12],
                'Timestamp': t['timestamp'],
                'Duration (ms)': t['duration_ms'],
                'Status': t['status'],
                'User': t.get('user_id', 'N/A')
            } for t in traces])

            # Select trace
            selected = st.dataframe(
```

```
                df,
                on_select="rerun",
                selection_mode="single-row"
            )

            if selected and len(selected['selection']['rows']) > 0:
                selected_idx = selected['selection']['rows'][0]
                trace_id = traces[selected_idx]['trace_id']

                # Display detailed trace
                self.render_trace_detail(trace_id)

    def render_trace_detail(self, trace_id):
        """Render detailed trace view"""
        st.divider()
        st.subheader(f"Trace Detail: {trace_id}")

        # Get full trace
        trace = self.db.get_trace_with_spans(trace_id)

        # Metadata
        col1, col2, col3 = st.columns(3)
        col1.metric("Duration", f"{trace['duration_ms']}ms")
        col2.metric("Status", trace['status'])
        col3.metric("Spans", len(trace['spans']))

        # Waterfall chart
        st.subheader("Latency Waterfall")

        waterfall_data = []
        for span in trace['spans']:
            waterfall_data.append({
                'Span': span['name'],
                'Start': span['start_time'],
                'Duration': span['duration_ms']
            })

        # Simple text-based waterfall (production would use plotly)
        for span in trace['spans']:
            bar_width = int(span['duration_ms'] / trace['duration_ms'] * 50)
            bar = '' * bar_width
            st.text(f"{span['name']:20s} {bar} {span['duration_ms']:6d}ms")

        # Span details
        st.subheader("Span Attributes")
```

```
        for i, span in enumerate(trace['spans']):
            with st.expander(f"{span['name']} ({span['duration_ms']}ms)"):
                # Attributes
                st.json(span['attributes'])

                # Events
                if span.get('events'):
                    st.write("**Events:**")
                    st.json(span['events'])

# Run Streamlit app
if __name__ == "__main__":
    db = TraceDatabase("postgresql://localhost/traces")
    viewer = TraceViewer(db)
    viewer.render()
```

Run with:

bash

```
streamlit run debugging/trace_viewer.py
```

Deliverable: Interactive trace viewer with:

- Filtering and search
- Waterfall visualization
- Detailed span inspection

Day 18-19: Sampling Strategies

Hands-on Lab:

python

```
# tracing/samplers.py
from opentelemetry.sdk.trace.sampling import Sampler, SamplingResult, Decision
import random
import hashlib

class IntelligentSampler(Sampler):
    """Smart sampling based on request characteristics"""

    def __init__(self, base_rate=0.1, error_rate=1.0, slow_rate=0.5):
        self.base_rate = base_rate
```

```
        self.error_rate = error_rate
        self.slow_rate = slow_rate
        self.p95_latency = 2000  # ms, updated periodically

    def should_sample(self,
                      parent_context,
                      trace_id,
                      name,
                      kind=None,
                      attributes=None,
                      links=None,
                      trace_state=None):
        """Decide whether to sample this trace"""

        # Always sample errors
        if attributes and attributes.get('error'):
            return SamplingResult(
                decision=Decision.RECORD_AND_SAMPLE,
                attributes=attributes
            )

        # Sample slow requests
        if attributes and attributes.get('latency_ms', 0) > self.p95_latency:
            if random.random() < self.slow_rate:
                return SamplingResult(
                    decision=Decision.RECORD_AND_SAMPLE,
                    attributes=attributes
                )

        # Sample flagged users
        if attributes and attributes.get('user_id') in self.get_flagged_users():
            return SamplingResult(
                decision=Decision.RECORD_AND_SAMPLE,
                attributes=attributes
            )

        # Base sampling rate for normal traffic
        if random.random() < self.base_rate:
            return SamplingResult(
                decision=Decision.RECORD_AND_SAMPLE,
                attributes=attributes
            )

        # Don't sample
        return SamplingResult(
            decision=Decision.DROP,
```

```
            attributes=attributes
        )

    def get_description(self):
        return f"IntelligentSampler(base={self.base_rate}, error={self.error_rate}, slow

    def get_flagged_users(self):
        """Users to always trace (VIPs, testers)"""
        return {'test_user', 'admin', 'vip_customer'}

class DeterministicSampler(Sampler):
    """Deterministic sampling based on trace_id hash"""

    def __init__(self, rate=0.1):
        self.rate = rate
        self.threshold = int(rate * 2**64)

    def should_sample(self, parent_context, trace_id, name, **kwargs):
        """Deterministic decision based on trace_id"""
        # Hash trace_id to number
        trace_hash = int(hashlib.md5(trace_id.to_bytes(16, 'big')).hexdigest(), 16)

        # Sample if hash below threshold
        if trace_hash % (2**64) < self.threshold:
            return SamplingResult(decision=Decision.RECORD_AND_SAMPLE)

        return SamplingResult(decision=Decision.DROP)

    def get_description(self):
        return f"DeterministicSampler(rate={self.rate})"

# Usage
from opentelemetry.sdk.trace import TracerProvider

# Apply sampler
sampler = IntelligentSampler(base_rate=0.05)  # 5% base rate
provider = TracerProvider(sampler=sampler)
```

Deliverable: Sampling implementation reducing costs by 80-90% while keeping critical traces.

Day 20-21: Portfolio Documentation

Create comprehensive README:

markdown

```
# Production LLM Observability System

## Overview
End-to-end observability for LLM applications with distributed tracing, metrics, and aut

## Architecture
```

LLM Request
↓
OpenTelemetry Instrumentation
↓
[Traces] → PostgreSQL → Trace Viewer
[Metrics] → Prometheus → Grafana
[Alerts] → Alert Manager → Slack/PagerDuty

```
## Features

### 1. Distributed Tracing
Multi-span tracing with parent-child relationships:
```python
Trace: rag_query (1.2s)
├─ retrieval (0.08s)
│  ├─ num_docs: 5
│  └─ top_score: 0.87
├─ reranking (0.15s)
│  └─ rerank_model: cross-encoder
└─ generation (0.95s)
   ├─ tokens_in: 1200
   ├─ tokens_out: 150
   └─ cost: $0.0021
```

2. Quality Metrics

Track LLM-specific quality:

- Hallucination rate: 3.2%
- Groundedness score: 0.94
- Relevance score: 0.89
- User satisfaction: 4.3/5
```

3. Automated Alerting

Smart alerts with playbooks:

Alert	Trigger	Playbook
High Error Rate	>10% errors	Classify errors, auto-remediate
Quality Degradation	Avg score <0.7	Analyze retrieval, check index
Cost Spike	>$50/hour	Enable rate limiting
Slow Queries	P95 >3s	Identify bottleneck spans

4. Error Classification

Automatic error categorization:

- Rate Limits: 35%
- Timeouts: 25%
- Validation Errors: 20%
- API Errors: 15%
- Other: 5%

5. Cost Optimization

Intelligent sampling:

- Always trace: Errors, slow requests, flagged users
- Sample 5%: Normal traffic
- **Result:** 90% cost reduction, 100% critical trace coverage

Results

Production Metrics (30 days)

- **Total Requests:** 5.2M
- **Traces Captured:** 780k (15% sample rate)

- **Avg Latency:** 950ms (P95: 2.1s)
- **Error Rate:** 2.3%
- **Avg Quality:** 0.91/1.0

Incident Response

- **MTTR (Mean Time To Resolution):** 15 min (was 2 hours)
- **False Alerts:** 2% (was 25%)
- **Auto-Resolved Incidents:** 45%

Debugging Efficiency

- **Trace-based debugging:** 85% of incidents
- **Manual log inspection:** 15% of incidents
- **Time saved:** 10x faster root cause analysis

Usage

Basic Instrumentation

```
from tracing import tracer

@tracer.trace_function("rag_query")
def rag_query(question):
    docs = retriever.search(question)  # Auto-traced
    answer = llm.generate(question, docs)  # Auto-traced
    return answer
```

Query Traces

```
from storage import TraceDatabase

db = TraceDatabase()

# Find slow traces
slow = db.get_slow_traces(threshold_ms=2000)

# Find errors
```

```
errors = db.get_error_traces(time_range=(start, end))

# Get specific trace
trace = db.get_trace_with_spans("abc123")
```

View Dashboard

```
# Start trace viewer
streamlit run debugging/trace_viewer.py

# View metrics
open http://localhost:3000/dashboards  # Grafana
```

Design Decisions

Why OpenTelemetry?

- Industry standard
- Vendor-neutral
- Rich ecosystem
- Future-proof

Tradeoff: More complex than proprietary solutions

Why Intelligent Sampling?

Random sampling misses critical errors.
Smart sampling captures 100% of errors at 10% of cost.

Tradeoff: Slightly more complex logic

Why PostgreSQL for traces?

- Rich querying (JSON support)
- ACID guarantees
- Mature ecosystem

Tradeoff: Not purpose-built for traces (vs Jaeger, Tempo)

Production Lessons

1. **Quality metrics are essential**

   • Latency alone doesn't tell you if LLM is working

   • Track hallucination, groundedness, relevance

1. **Sampling saves money**

   • 100% tracing → $1000/day

   • Intelligent sampling → $100/day

   • Same debugging capability

1. **Automated playbooks reduce MTTR**

   • Alert → Diagnose → Fix (automated)

   • 80% of incidents auto-resolved or pre-diagnosed

1. **Errors cluster by type**

   • Classifying errors enables systematic fixes

   • 35% rate limits → increase quota

   • 25% timeouts → optimize latency

   ```

   ```

---

4. Blind Spots & Underrated Perspectives

What's Poorly Taught (Industry Reality Gaps)

Blind Spot 1: Print Debugging Doesn't Scale to Production

Common teaching: "Just add print statements to debug."

Reality: Print debugging fails catastrophically in production LLM systems.

Why:

python

```
# Development (works fine)
def generate_answer(query):
    print(f"Query: {query}")
    docs = retriever.search(query)
    print(f"Retrieved {len(docs)} docs")
    answer = llm.generate(query, docs)
    print(f"Answer: {answer}")
    return answer

# Production (complete disaster)
# - 10,000 requests/minute
# - Logs fill disk in hours
# - No structure → can't query
# - No correlation → can't connect related events
# - No sampling → logs contain PII
# - No retention → old logs deleted
```

How this causes production incidents:

```
User Report: "Bot gave wrong answer at 3:42pm"

With print debugging:
1. Grep logs for timestamp → thousands of matches
2. Find relevant log lines → manual inspection
3. Correlate request → response → impossible (no trace_id)
4. Understand what happened → takes hours

With structured tracing:
1. Query by timestamp + user_id → get trace_id
2. View complete trace → all spans in one view
3. See exact inputs/outputs → immediate understanding
4. Root cause identified → 5 minutes
```

Strong engineer approach:

python

```
# Structured, traceable logging
with tracer.start_as_current_span("generate_answer") as span:
    span.set_attribute("query", query)

    docs = retriever.search(query)
    span.set_attribute("num_docs", len(docs))
```

```
    span.add_event("retrieval_complete", {"top_score": docs[0].score})

    answer = llm.generate(query, docs)
    span.set_attribute("answer_length", len(answer))

    return answer

# Now can query:
# - All requests from user X
# - All slow requests (duration > 2s)
# - All requests with low retrieval scores
# - Full context for each request
```

Blind Spot 2: Observability is Not Monitoring

Common teaching: "Set up Grafana dashboards and you're done."

Reality: Monitoring (metrics) and observability (traces) solve different problems.

Monitoring answers: "What is broken?"

```
Dashboard shows:
- Error rate: 15% (RED ALERT!)
- Latency P95: 3.2s (WARNING!)
- Request rate: 1000 req/s (NORMAL)

Insight: Something is broken
Action: ???
```

Observability answers: "Why is it broken?"

```
Trace shows:
├─ retrieval (50ms) ✓
├─ generation (2900ms) ❌
   └─ tokens_out: 2500 (!)

Insight: LLM generating too many tokens
Action: Add max_tokens=500 limit
```

When to use each:

Situation	Use Monitoring	Use Observability
"Is system healthy?"	✓	
"How many errors?"	✓	
"What's the P95 latency?"	✓	
"Why did this specific request fail?"		✓
"Where is latency coming from?"		✓
"What was the exact prompt that caused error?"		✓

Production reality: You need both.

python

```
# Monitoring: Aggregate metrics
prometheus.Counter("llm_errors_total").inc()
prometheus.Histogram("llm_latency_seconds").observe(duration)

# Observability: Individual traces
with tracer.start_as_current_span("request") as span:
    span.set_attribute("trace_id", trace_id)
    span.set_attribute("full_prompt", prompt)  # For debugging
    # ... execute request ...
```

Blind Spot 3: You Can't Debug What You Didn't Log

Common teaching: Not mentioned.

Reality: Most production debugging fails because critical information wasn't captured.

Example: Debugging hallucination

python

```
# What was logged (amateur):
{
    "timestamp": "2025-02-09T10:30:00Z",
    "user_id": "user_123",
    "error": "Hallucination detected"
```

```
}

# What engineer needs to debug:
# - What was the exact query?
# - What documents were retrieved?
# - What was the LLM's response?
# - What was the prompt?
# - What model/temperature was used?
# - What was the retrieval quality?

# All missing! Can't debug.
```

What to log (comprehensive):

python

```
with tracer.start_as_current_span("rag_query") as span:
    # Always log
    span.set_attribute("query", query)
    span.set_attribute("user_id", user_id)
    span.set_attribute("session_id", session_id)
    span.set_attribute("timestamp", time.time())

    # Retrieval
    docs = retriever.search(query)
    span.set_attribute("retrieval.num_docs", len(docs))
    span.set_attribute("retrieval.top_score", docs[0].score if docs else 0)
    # Store document IDs, not full text (PII concern)
    span.set_attribute("retrieval.doc_ids", [d.id for d in docs])

    # Generation
    prompt = build_prompt(query, docs)
    span.set_attribute("generation.prompt_template", "rag_v2")
    span.set_attribute("generation.model", "gpt-4")
    span.set_attribute("generation.temperature", 0)
    span.set_attribute("generation.tokens_in", count_tokens(prompt))

    # CRITICAL: Store prompt hash, not full prompt (PII)
    span.set_attribute("generation.prompt_hash", hash(prompt))
    # But store full prompt in secure storage for debugging
    secure_storage.store(prompt_hash, prompt, ttl=7*24*3600)

    answer = llm.generate(prompt)
    span.set_attribute("generation.tokens_out", count_tokens(answer))
```

```
    # Quality
    span.set_attribute("quality.groundedness", measure_groundedness(answer, docs))
    span.set_attribute("quality.hallucination_detected", groundedness < 0.8)

    return answer
```

Now can debug:

python

```
# User reports hallucination
trace = db.get_trace(user_id="user_123", timestamp="2025-02-09T10:30")

# Inspect
print(f"Query: {trace.attributes['query']}")
print(f"Retrieval quality: {trace.attributes['retrieval.top_score']}")
print(f"Groundedness: {trace.attributes['quality.groundedness']}")

# Get full prompt from secure storage
prompt_hash = trace.attributes['generation.prompt_hash']
full_prompt = secure_storage.get(prompt_hash)

# Root cause identified: Low retrieval score (0.42)
# Fix: Improve chunking strategy
```

Blind Spot 4: Privacy and Observability Conflict

Common teaching: Not mentioned.

Reality: Full observability requires logging prompts/responses, which often contain PII.

The problem:

python

```
# User query contains PII
query = "My name is John Smith, SSN 123-45-6789, how do I reset my password?"

# Logged in trace
span.set_attribute("query", query)  # ❌ GDPR violation!
```

Consequences:

- GDPR violations (fines up to €20M)

- SOC 2 compliance failure
- Customer trust issues
- Legal liability

Solutions:

1. Selective logging with PII detection

python

```
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine

analyzer = AnalyzerEngine()
anonymizer = AnonymizerEngine()

def log_safe_query(query):
    # Detect PII
    results = analyzer.analyze(text=query, language='en')

    if results:
        # Anonymize
        anonymized = anonymizer.anonymize(text=query, analyzer_results=results)
        span.set_attribute("query", anonymized.text)
        span.set_attribute("pii_detected", True)
    else:
        # Safe to log
        span.set_attribute("query", query)
        span.set_attribute("pii_detected", False)
```

2. Hash-based logging

python

```
# Don't log full content, log hash
query_hash = hashlib.sha256(query.encode()).hexdigest()
span.set_attribute("query_hash", query_hash)

# Store full query in encrypted storage with short TTL
encrypted_storage.store(query_hash, encrypt(query), ttl=24*3600)

# For debugging, retrieve by hash
if need_full_query:
    full_query = decrypt(encrypted_storage.get(query_hash))
```

3. Sampling for debugging

python

```
# Only store full prompts/responses for sampled traces
if should_sample_for_debugging(trace_id):
    # High-security storage with access logging
    debug_storage.store(trace_id, {
        'query': query,
        'prompt': prompt,
        'response': response
    }, ttl=7*24*3600)

    # Log who accessed it
    audit_log.log_access(trace_id, accessed_by=current_user)
```

4. User consent

python

```
# Let users opt into detailed logging
if user.debugging_consent:
    span.set_attribute("query", query)
else:
    span.set_attribute("query_hash", hash(query))
```

Blind Spot 5: Observability Has a Cost

Common teaching: "Trace everything!"

Reality: Comprehensive tracing can cost more than the LLM calls themselves.

Cost breakdown:

python

```
# Scenario: 1M requests/day, full tracing

# Trace storage cost
trace_size = 5 KB  # Average trace with all spans
storage_cost_per_gb = 0.023  # S3
traces_per_day = 1_000_000
daily_storage = traces_per_day * trace_size / 1024 / 1024 / 1024 * storage_cost_per_gb
# = 0.109 GB/day × $0.023 = $0.0025/day
monthly_storage = daily_storage * 30  # $0.075/month
```

```
# But: Storage accumulates!
yearly_storage = daily_storage * 365 = $0.92/year

# Query costs (assume 1000 queries/day)
query_cost_per_1000 = 0.005  # CloudWatch Insights
monthly_query_cost = 30 * query_cost_per_1000 = $0.15/month

# Egress costs (exporting traces)
egress_per_gb = 0.09  # AWS
monthly_egress = 3.27 GB * 0.09 = $0.29/month

# Total observability cost: ~$0.50/month

# Seems cheap, but:
# - This is just storage
# - Add processing (Langfuse, etc): +$50-200/month
# - At scale (10M requests/day): $5000/month
# - More than LLM API costs!
```

Smart engineer approach:

python

```
# Cost-aware observability
class CostAwareTracing:
    def __init__(self, monthly_budget=100):
        self.monthly_budget = monthly_budget
        self.current_spend = 0

    def should_trace(self, request):
        # Always trace important stuff
        if request.is_error or request.is_slow:
            return True

        # Check budget
        trace_cost = 0.0001  # $0.0001 per trace

        if self.current_spend + trace_cost > self.monthly_budget:
            # Budget exceeded - reduce sample rate
            return random.random() < 0.01  # 1% sample

        # Normal sampling
        return random.random() < 0.1  # 10% sample
```

How This Perspective Prevents Failures

Fragile Observability (Amateur):

- Print statements for debugging
- No structured logging
- No trace correlation
- No sampling (logs everything or nothing)
- No PII handling
- **Result:** Can't debug production issues, GDPR violations, high costs

Robust Observability (Engineer):

- OpenTelemetry distributed tracing
- Structured spans with rich metadata
- Intelligent sampling (5% base, 100% errors)
- PII detection and anonymization
- Cost-aware trace retention
- **Result:** Fast debugging, compliant, cost-effective